TECHHAVEN STUDIOS

HAVENDOS ROAD TO 1.0

The release plan from v0.8.3.1 to 1.0, and the web at 1.1
Fifteen releases, decided question by question

BLOCK LAYER	##################....	ATA, VirtIO, AHCI, NVMe r/w
CONTROLLERS	#######..............	UHCI only, no xHCI
NETWORK	######...............	RTL8139 and 82540 only
TRANSPORT	##########...........	TLS 1.2, one RSA suite
DISPLAY	########.............	1280x800, 80x25 grid
APPLICATIONS	####.................	45 apps, every one at 80x25

DOCUMENT	THS-RM-1003 REV A · 19 OCT 2026
AUTHOR	Kyan Hoekstra / TechHaven Studios
BASELINE	v0.8.3.1 “Lodger”, 1657 checks across 26 suites
SUPERSEDES	THS-RM-1002 in full
STATUS	PROPOSAL — decided 19 OCT 2026, not yet started

Contents

1 What changed since THS-RM-1002 4
2 Ground truth after 0.8.3.1 7
3 What 1.0 means, and what 1.1 means 9
4 How the releases are written 10
5 The release map 11
6 0.8.4 “Hermes” — the update path 13
7 0.8.5 “Atlas” — xHCI, the USB core and HID 15
8 0.8.6 “Argo” — USB mass storage 18
9 0.8.7 “Iris” — three network adapters 20
10 0.8.8 “Helios” — any grid, chosen at boot 23
11 0.8.9 “Daedalus” — app port, wave one 25
12 0.9.0 “Hephaestus” — app port, wave two 26
13 0.9.1 “Gaia” — the end of the 2 GB cap 28
14 0.9.2 “Prometheus” — a boot entry, and self-hosting 30
15 0.9.3 “Orpheus” — audio, and the player unparked 32
16 0.9.4 “Midas” — the trackpad, if it is I2C 34
17 0.9.5 to 0.9.9 — stabilisation 36
18 1.0 “Olympus” 38
19 1.1 “Aegis” and 1.2 “Janus” — the web 39
20 Cross-cutting work 41
21 Sequencing 43
22 Release gates 44
23 Deliberately not in 1.0 46
24 After 1.2 — the horizon 47
25 Open questions 49

26 How all of it is verified 50

1 What changed since THS-RM-1002

RM-1002 planned ten releases from v0.8.2 to 1.0 and was approved on 12 October 2026. Two of them have shipped, four things have been learned since, and one of those four moves a whole arc of the plan out of 1.0. This document replaces RM-1002 rather than amending it.

The numbers came apart from the map

RM-1002's **0.9.0 "Ledger"** — the additive GPT and ESP installer — shipped as **v0.8.3 "Lodger"**. Its **0.9.1 "Anchor"** — the NVMe write path — shipped as **v0.8.3.1**, along with AHCI, the MMIO test seam and six bugs found in one afternoon on real hardware. Its **0.9.3 "Spindle"** is substantially done too: AHCI exists, and the ATA driver no longer stops probing at the first drive it finds.

So three of ten are delivered and the version numbers no longer point at the document. Every release below is renumbered from 0.8.4 upward, in dependency order.

The test laptop has no Ethernet

> **FINDING 1**
>
> The HP's only network adapter is a MediaTek 14C3:7902, and it is **Wi-Fi**. There is no RJ45 socket on the machine. Wi-Fi needs signed firmware blobs per chipset and a full 802.11 MAC, which RM-1002 ruled out of reach for a solo bare-metal kernel and this document does not reopen. **The only way that laptop reaches a network is a USB Ethernet adapter** — which means xHCI, which means the USB core.

This is the single finding that most changes the shape of the plan. USB is no longer one feature among several: on the one machine this project is actually aimed at, USB gates **input**,

removable storage and **the network**. It becomes the spine, and four consecutive releases hang off it.

There is no SATA machine in the house

> **FINDING 2**
>
> The ThinkPad X140e used through v0.7.11 belonged to a friend and is no longer available. The HP has **no SATA controller at all**. drivers/ahci.c therefore cannot be proved on real silicon by anyone working on this tree, and will ship at 1.0 having run only against the simulated controller in tests/test_ahci.c and QEMU's q35.

The decision taken is to **ship it and say so in writing**, in the release notes and in the 1.0 claims, rather than quietly imply hardware coverage that does not exist. Section 22 carries the qualification.

The browser is gated on cryptography, not HTML

> **FINDING 3**
>
> net/tls.c offers exactly one cipher suite: TLS_RSA_WITH_AES_128_CBC_SHA256 (0x003C). RSA key exchange was removed in TLS 1.3 and most public servers stopped accepting it years ago. A browser built on what the tree has today would reach a shrinking handful of sites; an HTTP-only browser in 2026 reaches almost nothing at all.

The work the browser actually needs is X25519, AES-GCM, HKDF, the TLS 1.3 handshake, signature verification and a trust store — none of which exists. RM-1002 called that 1.1 and was right. The browser follows it.

There is no upgrade path to the test machine

fs/update.c reads the new image from an **ATAPI CD-ROM**. The HP has no optical drive, no USB mass storage support in HavenDOS and no supported NIC, so every one of the fourteen releases below would otherwise reach it by full reinstall — which means running the additive installer beside a live Windows partition, fourteen more times. That is why section 6 comes first and nothing else does.

2 Ground truth after 0.8.3.1

What the tree actually contains on 19 October 2026, read rather than remembered. Everything in section 5 is scoped against this table and nothing else.

Layer	Has	Has not
Block	ATA/IDE with LBA48, ATAPI, AHCI, NVMe read and write, VirtIO	USB mass storage. SD, eMMC, SAS, RAID, Intel VMD
Filesystem	FAT12, FAT16 read/write/format, FAT32 read, ISO9660, GPT read/write, ESP writer, ramfs, VFS	A FAT32 *formatter* — so every partition stops at 2047 MB. No NTFS, no ext
USB	UHCI: control and bulk transfers, port reset, enumeration, 930 lines, 110 checks	xHCI, EHCI. No HID class, no mass-storage class, no USB networking
Network	RTL8139, Intel 82540 (e1000), ARP, IPv4, ICMP, UDP, TCP with six sockets and retransmission, DHCP, DNS with cache, HTTP	Every NIC made after about 2008. RTL8169/8168, e1000e, I219
Transport security	TLS 1.2 with one RSA/CBC suite, MAC verification, bounds-checked certificate parsing	ECDHE, AES-GCM, TLS 1.3, signature verification, a trust anchor store
Display	VGA text modes, a 1280x800 framebuffer console, VBE, mode 13h, an 8x8 font with a CP437 patch table	Any grid other than the compiled one. No panel auto-detection
Input	PS/2 keyboard and mouse	USB HID. I2C-HID. The HP has no PS/2 port of either kind
Audio	PC speaker	AC97, HD Audio. The music player is parked behind all of it

Layer	Has	Has not
Kernel	PMM, heap, PIT, RTC, IDT with working frame pointers since 0.8.3.1, entropy pool, klog	A scheduler, processes, memory protection, UEFI runtime services

> **NOTE**
> The IDT fix in 0.8.3.1 matters more than its changelog entry suggests. Neither interrupt stub had ever passed the frame pointer to its handler, so irq_handler was selecting a function pointer by an index read out of low memory about a hundred times a second. Nothing depended on interrupts, because HavenDOS polls, so nothing reported it. Audio is the first thing in this plan that genuinely needs an IRQ — include/uhci.h says so in its own header comment — and it now has one that works.

3 What 1.0 means, and what 1.1 means

RM-1002's sentence stands unchanged. What changed is where the web sits relative to it.

1.0

HavenDOS installs itself onto a computer made in the last twenty years — whatever the disk is attached to — boots from it under BIOS or UEFI, fills the screen it is given, reaches the network, takes input, and does so without damaging whatever was on the machine beforehand.

Six testable claims: **installs, boots, fills the screen, networks, takes input, harms nothing.** Section 22 turns each into something observable on hardware.

1.1

HavenDOS browses the web. Modern TLS first — X25519, AES-GCM, TLS 1.3, signature verification, a trust store — then a text browser on top of it.

Splitting them this way was a decision, not an accident. 1.0 is a hardware release and it can be finished; the web arc is a second body of work of comparable size, and binding the two together means neither ships. The browser is not cut — it is section 19, written out in the same detail as everything before it.

"Every possible drive" is scoped honestly as IDE/PATA, SATA through AHCI, NVMe, VirtIO and USB mass storage — with the AHCI qualification from Section 1 attached. SD, eMMC, SAS, hardware RAID and Intel VMD are out, and section 23 says why.

4 How the releases are written

Each release below has the same five parts, so they can be compared and picked up in whatever order the dependencies allow.

Part	What it answers
GOAL	What a person can do after it that they could not before.
WORK	Numbered items. Each is a sitting of work or close to it.
TEST	How it is proved. A suite that cannot fail proves nothing.
DONE	The acceptance check. Something observable, not “it compiles”.
RISK	What could go wrong and what to do about it.

The patch-release rule

> **DECIDED**
>
> Every feature release is followed onto hardware, and whatever the hardware finds ships as a **patch release** — 0.8.5 then 0.8.5.1, exactly as 0.8.3 was followed by 0.8.3.1. Fixes are not folded silently into the next feature.

The reason is on the record: 0.8.3.1 fixed six bugs found in a single afternoon on metal, and **five of them had survived 1636 checks and eight injection harnesses**. If those six had been folded into the next feature release, the next regression would have had two causes to bisect between instead of one. A patch release is also permission to ship a feature before it is perfect, which is what keeps the fourteen below from becoming four.

A patch release carries no new feature and no version-number ambiguity: HAVENDOS_VER_NUM is the single source of truth, tools/check_iso_version.py refuses a build whose ISO and kernel disagree, and that check stays green through a patch exactly as through a feature.

5 The release map

Release	Name	Unlocks
0.8.3.1	Lodger	Additive UEFI install, AHCI, NVMe write — done
0.8.4	Hermes	Update from the ESP — new versions reach the laptop
0.8.5	Atlas	xHCI, the USB core, USB HID — keyboard and pointer
0.8.6	Argo	USB mass storage — install from and to a stick
0.8.7	Iris	USB Ethernet, RTL8169/8168, e1000e/I219
0.8.8	Helios	Any grid, auto-detected, plus the viewport layer
0.8.9	Daedalus	App port wave 1 — the screens nobody can skip
0.9.0	Hephaestus	App port wave 2 — the remaining ~35 apps
0.9.1	Gaia	FAT32 formatter — the end of the 2047 MB cap
0.9.2	Prometheus	UEFI boot entry, and a system that can install itself
0.9.3	Orpheus	AC97 and HD Audio — the music player unparked
0.9.4	Midas	I2C-HID trackpad, if the measurement says it is needed
0.9.5–0.9.9	—	Fuzzing, soak, hardware matrix, manual, freeze, RC
1.0	Olympus	The six gates, on hardware
1.1	Aegis	Modern TLS — X25519, AES-GCM, TLS 1.3, a trust store
1.2	Janus	The browser

Fourteen releases to 1.0 and two beyond it, each shipping a bootable ISO, a source archive and a changelog, as every milestone in this project has. Plus a patch release after any of them that hardware disagrees with.

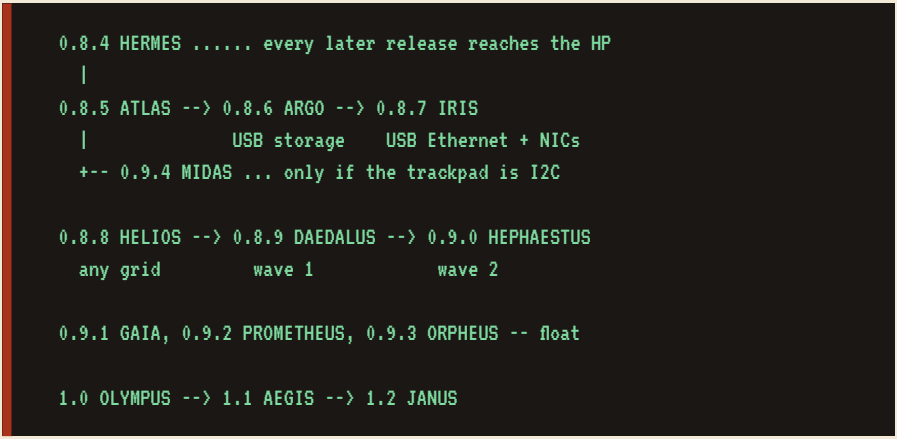


FIGURE 5-1 What blocks what. The USB chain is strictly ordered. The screen chain is strictly ordered. Gaia, Prometheus and Orpheus can move anywhere after 0.8.6. Nothing in the web arc can start before the crypto in 1.1.

NOTE

The order puts the HP first without making other people’s machines an afterthought: it **updates** at 0.8.4, **takes input** at 0.8.5, **boots from a stick** at 0.8.6, **reaches a network** at 0.8.7 and **looks right** at 0.8.8. Breadth for desktops arrives inside 0.8.7 with the two Realtek and Intel gigabit families, and at 0.9.1 with partitions larger than 2 GB.

6 0.8.4 “Hermes” — the update path

SIZE	Small
DEPENDS	Nothing. The FAT32 reader and the ESP writer both exist
UNLOCKS	Every release after this one, on the only real machine
RISK	Low. It reads from a partition it already knows how to read

GOAL. A new HavenDOS version is installed by copying one file onto the EFI System Partition from Windows and running update — no optical drive, no reinstall, and no second pass of the additive installer beside a live Windows partition.

WORK.

1. An **ESP source** for fs/update.c. The existing code is four steps bound to ATAPI: detect the drive, mount ISO9660, read /version.txt, stream /boot/havendos.elf. Split that into a source interface with two implementations — the existing ATAPI one, and a new one reading \EFI\HAVENDOS\update\ off the FAT32 ESP. **Do not add a second block layer.** The ESP is found the way the installer finds it, by partition type GUID.
2. A **manifest** beside the kernel: version, byte length, and a checksum of the image. update verifies the checksum before it overwrites anything, which the ATAPI path never did — it trusted the ISO.
3. **Source auto-selection.** Prefer the ESP when it carries a newer manifest; fall back to ATAPI; say which one it chose on screen. The bus label lesson from 0.8.3.1 applies — one function decides, every screen asks it.
4. The **ESP_E_BACKUP fix.** esp_install() stops when BOOTX64.HDB already exists rather than deciding which of two backups matters. If the current BOOTX64.EFI is already our own loader, there is nothing worth backing up: skip the backup and carry on.

5. A short rescue/ note and a PowerShell helper that drops a new image into the ESP from Windows with mountvol, so the whole route is one documented command rather than folklore.

TEST.

- A new suite over the manifest parser and the version comparison, with an injection harness. ver_cmp() has shipped a bug before — fs/update.c once carried a hard-coded "0.5.9.14" and compared every ISO against it, so an *older* image applied as an upgrade.
- A model ESP built by the test, with the image present, absent, truncated, and checksum-wrong. Each must produce its own message.
- In QEMU under OVMF: install, write a newer image to the ESP from the host with mtools, boot, update, reboot, confirm the version.
- Byte-identity on the ESP before and after: the new file is added, and no existing file is touched except the kernel the update replaces.

DONE

On the HP: copy one file into the ESP from Windows, boot HavenDOS, run update, and the machine comes back up on the new version with Windows still booting. A second install into the same gap completes without hand-restoring BOOTX64.EFI.

CAUTION

update overwrites a kernel on a disk that has Windows on it. The checksum is verified **before** the first byte is written, not after, and the old kernel is kept beside the new one until the new one has booted once.

7 0.8.5 “Atlas” — xHCI, the USB core and HID

SIZE	Large — the largest release before 1.0
DEPENDS	The MMIO seam, which 0.8.3.1 delivered
UNLOCKS	Input on a machine with no PS/2 port. And 0.8.6, and 0.8.7
RISK	xHCI bring-up is intricate and the HP is the only test target

GOAL. HavenDOS enumerates devices on a USB 3 controller and takes keyboard and pointer input from them. On the HP — which has 8086:51ED and no UHCI whatsoever — that is the difference between a usable machine and one that cannot be typed at.

The USB core comes first, and it is the real work

ARCHITECTURE

drivers/uhci.c is controller-shaped all the way up: the device struct is uhci_dev_t, the transfers are uhci_control() and uhci_bulk(), and the enumeration walk lives inside the driver. Adding xHCI beside it would produce two device models, two enumeration walks, and two places that choose a bus — **which is exactly the mistake that cost 0.8.3.1 three of its six bugs.**

So this release introduces usb/core.c: one device table, one descriptor walk, one place that assigns addresses and chooses configurations, and a host-controller interface with **two** back-ends behind it from the day it is born. UHCI is refactored onto it in the same release, where its 110 existing checks catch any mistake, and only then is xHCI written. A registry born with one user is a wrapper; born with two it is an interface.

WORK.

1. **usb/core.c and include/usb.h.** usb_dev_t, the standard descriptor walk, address assignment, configuration and interface selection, endpoint records, and usb_control() / usb_bulk() / usb_interrupt() dispatching to a back-end.
2. **UHCI moved behind it**, with its existing suite green and its injection harness re-run. No behaviour change is permitted in this step; it is a refactor and it is proved as one.
3. **xHCI bring-up:** BAR0 and the capability registers, the operational registers, CRCR and the command ring, DCBAA, the device context array, the event ring and its segment table, port registers, and a reset that waits on CNR rather than on a delay.
4. **xHCI transfers:** Enable Slot, Address Device, Configure Endpoint, control transfers over the default pipe, bulk and interrupt endpoints, transfer rings with cycle-bit handling, and the event ring polled. **Polled, as everything else is** — the house rule that no I/O depends on an interrupt stands.
5. **USB HID:** boot-protocol keyboard and boot-protocol mouse. Set Protocol to boot, poll the interrupt IN endpoint, and feed the existing keyboard and mouse queues so nothing above the driver changes. Boot protocol is a fixed 8-byte and 3-or-4-byte report; the full report descriptor parser is not in this release and is not needed for it.
6. **A device view.** usb -vv already exists; it now lists every device on every controller with its class, speed, endpoints and chosen configuration. This is also the instrument that answers section 25 about the trackpad.

TEST.

- A **simulated xHCI controller** that executes commands, in the shape of tests/test_ahci.c and tests/test_nvmerw.c — rings that advance, cycle bits that flip, events that are posted. A mock returning success proves nothing and the tree has said so since 0.8.2.

- An injection harness on the core *and* on xHCI. Every harness written so far has found holes; assume this one does too.
- The UHCI refactor proved by its own unchanged suite, run before and after.
- A descriptor-walk suite with real captured descriptors: a hub, a composite device, a device with more endpoints than the table holds, and a device that STALLs the configuration request.

DONE

A USB keyboard types at the HavenDOS shell prompt on the HP, with no PS/2 port in the machine, and usb -vv lists the devices actually plugged in rather than an invented list — the check_boot_log.py rule about invented output applies to this screen.

CAUTION

xHCI is the one release here with no fallback. If it does not come up on the HP, 0.8.6 and 0.8.7 have nothing to stand on, and the laptop has no input, no removable storage and no network. Budget a patch release for it from the start rather than discovering the need.

8 0.8.6 “Argo” — USB mass storage

SIZE	Medium
DEPENDS	0.8.5 — bulk transfers need endpoints, which need the walk
UNLOCKS	Install from a stick, install to a stick, update from a stick
RISK	A new bus in the block layer, which has bitten this tree twice

GOAL. A USB stick is a disk. It can be read, written, installed from, installed to, and updated from. On a laptop with no optical drive this is the first removable medium HavenDOS has ever had.

WORK.

1. **Bulk-Only Transport.** Command Block Wrapper out, data in or out, Command Status Wrapper in; tag matching, residue handling, and the Get Max LUN request.
2. **SCSI over BOT:** INQUIRY, TEST UNIT READY, READ CAPACITY (10 and 16), READ (10), WRITE (10), REQUEST SENSE. Plus the retry a real stick needs on UNIT ATTENTION after enumeration, which is the single most common reason a first read fails.
3. **Stall recovery:** Clear Feature HALT on the offending endpoint, then Bulk-Only Mass Storage Reset if that is not enough. Without it one bad command ends the session.
4. **Registration as a block device**, through whatever fs/fat16.c, fs/fat32.c and the installer use to choose a bus — **all of them, in one commit.**
5. **Removable-media semantics:** media present, media changed, and an installer that refuses a stick it is currently booted from.

THE RULE THIS RELEASE EXISTS TO HONOUR

0.8.3.1's own post-mortem: *"the installer has one block layer and the filesystem has another, and only one of them got taught."* Three of its six bugs were that mistake. **When you add a bus, grep for every place that chooses one** — and this release adds a bus. Expect at least four call sites and prove each of them with a test that reaches it.

TEST.

- A simulated BOT device that executes SCSI commands, including the failure paths: short transfers, phase errors, stalls, UNIT ATTENTION, and a device that reports a capacity it cannot deliver.
- An injection harness over BOT and over the SCSI layer.
- A bus-selection suite in the shape of tests/test_fat16nvme.c: the model disk reachable **only** over USB, every other bus reporting nothing, and the filesystem required to find it anyway.
- In QEMU, with -device usb-storage: mount, read, write, fsck.vfat clean afterwards, and an install to the stick that boots.

DONE

A FAT16 stick plugged into the HP is listed by disk, mounted, read and written; and an update image placed on that stick from Windows upgrades the machine.

9 0.8.7 “Iris” — three network adapters

SIZE	Large — three drivers, one of them over USB
DEPENDS	0.8.5 for the USB adapter; nothing for the two PCI ones
UNLOCKS	A network on the HP at all, and on most real desktops
RISK	Three chip families, none of which can be fully tested here

GOAL. HavenDOS gets a DHCP address on hardware people actually own. Today it drives an RTL8139 and an Intel 82540 — both of them QEMU parts, neither of them in a machine made this decade.

Why three, and why these three

Adapter	Reaches	Why it is in
USB Ethernet	The HP, and every thin laptop with no RJ45	The only route to a network on the one machine this plan is aimed at
RTL8169 / 8168	Most consumer desktops and older laptops	The commonest gigabit NIC there is. RTL8139 is its ancestor, not it
Intel e1000e / I219	The other half of real desktops, 2008 onward	The existing e1000 driver covers the 82540 only — a QEMU part

virtio-net was considered and left out. It is cheap to add and helps testing, but it helps no physical machine, and this release is specifically about physical machines.

WORK.

1. **A NIC interface** in net/: open, close, MAC address, send a frame, poll for a frame, link status. Three drivers land at once, so the interface is born with three users and the lesson from 0.8.5 is applied rather than relearned. net/ethernet.c stops naming a chip.

2. **USB Ethernet:** CDC-ECM first, because it is the standard and the simplest; then ASIX AX88179 and Realtek RTL8153, which is what most cheap dongles actually are. Bulk IN for receive, bulk OUT for transmit, the interrupt endpoint for link status.
3. **RTL8169/8168:** descriptor rings in main memory, the OWN bit, transmit and receive poll registers, and the per-revision quirks table — this family has many revisions and they differ.
4. **Intel e1000e / I219:** the same descriptor-ring shape as the existing e1000, with the newer register layout, PHY access over MDIC, and the device-ID table that separates them.
5. **Driver selection:** one probe that walks PCI and USB, picks the first adapter it can drive, and reports which one on screen and in ifconfig. One function decides, every screen asks it.
6. **The networking backlog**, which has been open since v0.7.15 and costs little once the stack is being touched: DHCP lease renewal, HTTP POST, chunked transfer, keep-alive, wget progress and resume, and traceroute. Also dns_resolve(), which still returns 0 both for a failed lookup and for the address 0.0.0.0.

TEST.

- A simulated descriptor-ring NIC that moves frames, with an injection harness, shared by the two PCI drivers.
- In QEMU, each PCI driver against its emulated part where one exists; the full stack exercised by ping, nslookup, wget and BOOT Chat.
- A loopback test for the USB Ethernet path that does not need the hardware: BOT-style framing over the simulated xHCI.
- On the HP with a real dongle: ifconfig shows a DHCP lease from the actual router.

DONE

The HP, with a USB Ethernet adapter plugged in, gets a DHCP address and BOOT Chat connects — which is the 1.0 **networks** gate, met on the machine that could not meet it any other way.

NOTE

Buy the dongle before starting this release and check what chip it contains. A £8 adapter is usually RTL8153 or AX88179; CDC-ECM-only parts are rarer than the specification would suggest. The driver written first should be the one in the adapter on the desk.

10 0.8.8 “Helios” — any grid, chosen at boot

SIZE	Medium
DEPENDS	Nothing. Can be done in parallel with the USB chain
UNLOCKS	0.8.9 and 0.9.0, and the **fills the screen** gate
RISK	A console that will not sync leaves no way to read a prompt

GOAL. HavenDOS asks the firmware what the panel is, picks the largest grid that fits it, and renders the console at that size. On a 1080p laptop that is roughly 240x67 cells at 8x16, not 80x25 centred in the middle of the screen.

WORK.

1. **Mode discovery.** Take what GRUB hands over with gfxpayload=keep, and enumerate VBE modes as a fallback. Choose by panel size, not by a compiled constant.
2. **A console sized at runtime.** vga_rows() and vga_cols() already exist and the desktop chrome already derives from them since v0.7.18. The shadow buffer, fbc_painted[], the scroll stride, the cursor wrap and the tab stops all become dynamic. **v0.7.12 found five sites pinned to the compile-time 80-column constant**; assume there are more and grep for every one.
3. **The viewport layer** — a push and pop that centres a sub-grid inside the real one and offsets every positioned draw. One flag per entry in the app registry says whether an app has been ported.
4. **A confirm-or-revert timer** on any user-chosen mode, as v0.7.10 established. And remember get_ticks() counts 10 ms units — that has been got wrong twice, once on this exact timer, where a “10 second” revert was really 100 seconds.
5. **An 8x16 font option.** At 1920x1080 an 8x8 cell gives 240x135, which is unreadable. The font work must not repeat the v0.7.9 trap where uploading an 8x8 font to VGA plane 2 destroyed the BIOS 8x16 one.

WHY THE VIEWPORT IS NOT OPTIONAL

Without it this release is a **regression**. A 240x67 console with 45 apps still drawing at 80x25 puts every one of them in the top-left corner of a very large screen, which is worse than the centred box they sit in today. The viewport is what makes 0.8.9 and 0.9.0 safe to do app by app instead of as one 550-site commit.

TEST.

- A geometry suite over mode selection: 1920x1080, 1366x768, 1280x800, 1024x768, 800x600, and a firmware that reports something absurd.
- A viewport suite: a draw at (0,0) inside an 80x25 viewport on a 240x67 grid lands at the centred offset, and a clipped draw clips.
- harness/fbtext.py reads the screen back by decoding pixels against the kernel's own font — it must keep working at every new grid size.
- tools/check_screen_width.py currently refuses a string literal past column 80. It stays: 80 columns remains the minimum supported width, and a wider screen must not become an excuse for text that breaks on a narrow one.

DONE

The HP boots to a console that fills its panel, at a grid nobody told it to use, and every un-ported app still looks exactly as it does today.

11 0.8.9 “Daedalus” — app port, wave one

SIZE	Medium
DEPENDS	0.8.8 — there is no larger grid to port to without it
UNLOCKS	The screens a new user cannot avoid seeing
RISK	Low, and bounded: an app that fights back keeps its viewport

GOAL. The screens nobody can skip use the whole display: the desktop, the shell, the installer, the first-boot setup wizard, the file manager, help and man.

WORK. Each app in turn: replace absolute 80x25 coordinates with layout derived from vga_cols() and vga_rows(), clear its ported flag in the registry, and check it at 80x25, 120x40 and 240x67. The pagers in help and man need their page height derived rather than assumed — they are also the thing that breaks scripted testing, so fixing their geometry is worth more than it looks.

TEST. A screenshot of each ported app at three grid sizes, read back with harness/fbtext.py and checked for the frame reaching the edges. The desktop menu’s two-panel marker layout must be read by column, as the test notes already say.

DONE

A first boot on the HP — installer, setup wizard, desktop, shell — never shows an 80x25 box in the middle of a 1080p screen.

12 0.9.0 “Hephaestus” — app port, wave two

SIZE	Medium, and long rather than hard
DEPENDS	0.8.9
UNLOCKS	The **fills the screen** gate, completely
RISK	Tedium. Thirty-five apps is thirty-five chances to be careless

GOAL. The remaining ~35 apps fill the grid: the clock, the dashboard, the monitor, the games, the screensavers, the chat client, neofetch, the package manager, the BOOT language editor and the rest.

WORK. The same recipe as 0.8.9, app by app, in order of how often anyone opens them. An app that genuinely wants a fixed aspect — Tetris, mode 13h things — **keeps its viewport on purpose** and the registry records that it is finished rather than pending.

THE CONTRACT THAT BREAKS FIRST

Every one of these apps is a draw-poll-sleep loop, and the framebuffer console has exactly two commit points: keyboard_waitchar() flushes before it blocks, and sleep_ms() calls vga_present(). An app that polls without sleeping never commits a frame. Touching 35 draw loops is the most likely way to break that contract, and it was broken once already.

TEST. The same three-grid screenshot check, plus a flicker check: put_cell() must skip a cell the screen already shows, in both branches. The asymmetry between them was the whole of the flicker in 0.8 Beta.

DONE

Every entry in the app registry is marked ported or deliberately fixed, and nothing in the desktop opens into a corner.

13 0.9.1 “Gaia” — the end of the 2 GB cap

SIZE	Medium
DEPENDS	Nothing. The FAT32 reader has been proved since 0.8.2
UNLOCKS	A HavenDOS partition the size of the disk it is on
RISK	It writes filesystem metadata. Byte-identity or nothing

GOAL. A HavenDOS partition can be any size. Today fat16_geom() stops at 65524 clusters of 64 sectors — **2047 MB** — so a 512 GB SSD gets a 2 GB HavenDOS and 510 GB of nothing, which is also what caused the E15 failure that 0.8.3.1 fixed.

WORK.

1. **fs/fat32fmt.c**, in the shape of fs/fat16fmt.c: BPB and the FAT32 extensions, FSInfo, the backup boot sector at sector 6, two FATs, a root directory that is an ordinary cluster chain, and cluster sizing by volume size as Microsoft’s table specifies.
2. **The installer offers FAT32** above a threshold and FAT16 below it, with the geometry computed and validated **before the first sector is written** — the rule 0.8.3.1 established after a failed install left a disk unbootable at step 4 of 9.
3. **The FAT32 write path**, which the reader does not have: directory entry creation, cluster allocation through FSInfo, and long filenames.
4. **GRUB and the boot chain** on a FAT32 root — the search --label HAVENDOS needs the root directory entry with attribute 0x08, not the BPB copy. That exact mistake has already been made once on FAT16.
5. **The FAT16 dotfile bug**, carried since before 0.8 and now in two formatters’ worth of code: name_to_83() turns .desktop into a blank basename with extension DES. Nine files are affected. The FAT32 driver’s 8.3 check is the shape the fix should take.

TEST. fsck.vfat judges every image, as it has since 0.8.2 — our code agreeing with itself is not evidence. A size sweep in the shape of tests/test_wholedisk_size.c, from the FAT32 minimum to 2 TB, asserting a valid geometry, a partition that fits, megabyte alignment, and an MBR or GPT entry describing the same extent the filesystem occupies. An injection harness on the formatter; the FAT16 one found 6 holes in 21.

DONE

An install to a 512 GB disk offers a partition larger than 2047 MB, formats it, boots from it, and fsck.vfat reports it clean.

14 0.9.2 “Prometheus” — a boot entry, and self-hosting

SIZE	Medium
DEPENDS	Nothing, though it is more useful after 0.8.6
UNLOCKS	Installing without Windows, and installing from HavenDOS
RISK	UEFI runtime services from a 32-bit kernel is the hard part

GOAL. Two things that both read as “the installer finished but you are not done yet”.

A real firmware boot entry

HavenDOS writes the removable-media fallback \EFI\BOOT\BOOTX64.EFI, and firmware with an existing boot order does not look for it. On the HP the entry had to be added **from Windows** with bcdedit /copy {bootmgr} pointed at \EFI\HAVENDOS\grubx64.efi. A machine that needs another operating system to become bootable is not installed. The fix is a real NVRAM entry, which needs UEFI runtime services — SetVariable on Boot#### and BootOrder — reached from a 32-bit kernel, which is the genuinely awkward part and should be investigated before the release is scheduled.

A system that can install itself

The two-pass build means pass 1 produces a kernel with no embedded image and pass 2 embeds *that* one, so the kernel the installer writes to a disk carries no image of its own. **An installed HavenDOS cannot install HavenDOS.** Before 0.8 this was discovered at step 7 of 9, after the disk had been repartitioned and formatted; havendos_install_image_available() is a precondition now, so it refuses safely — but it still refuses. The fix is a streaming install that reads the image from the

running system’s own boot partition or from removable media rather than from a copy compiled into the kernel.

DONE

A fresh install on the HP appears in the firmware boot menu without touching Windows; and an installed HavenDOS can install HavenDOS onto a second disk.

15 0.9.3 “Orpheus” — audio, and the player unparked

SIZE	Large
DEPENDS	0.8.6 — the player reads its files off a stick
UNLOCKS	The music player project, parked since 11 October 2026
RISK	The first thing in this kernel that genuinely needs an IRQ

GOAL. HavenDOS plays audio. The MP3-from-USB player that was parked behind the road to 1.0 becomes buildable, because the storage half of it arrived at 0.8.6.

A DECISION THE TREE ASKED TO BE MADE OUT LOUD

include/uhci.h says it in its own header: *“Audio will be the first thing in this kernel that genuinely needs an IRQ, and that is a decision to make out loud when it arrives, not to smuggle in here.”* It has arrived. A codec consuming a ring buffer in real time cannot be driven by a polling loop that also has to draw a screen. **Audio takes an interrupt; nothing else does.** The IDT stubs were only fixed in 0.8.3.1, and UTM SE’s TCG does not deliver IRQs reliably at all — so the driver must degrade to polled playback with a larger buffer when the interrupt does not arrive, and say which mode it is in.

WORK.

1. **AC97** — the simpler of the two, and what QEMU emulates: the mixer, bus-master descriptor lists, PCM out, and the interrupt on buffer completion.
2. **HD Audio (Intel HDA)** — what every real machine since about 2005 actually has, the HP included: CORB and RIRB, the codec walk, widget discovery, stream descriptors and BDL. Larger than AC97 and the reason this release is marked Large.

3. **A mixer and an audio device interface**, so the two chips look the same from above and whatever comes third does too.
4. **WAV playback**, then **MP3 decode** — fixed-point, because there is no FPU use anywhere in this kernel and introducing one here would be a surprise.
5. **The player UI**, written for the grid the machine has rather than for 80x25, since 0.8.8 has landed by now.

TEST. A simulated codec that consumes a buffer and raises completion, with an injection harness. Decoder correctness against known-good PCM produced by ffmpeg on the build host — our code agreeing with itself is not evidence. Playback in QEMU with -device AC97; remember the audiodev must be declared before the device or QEMU refuses.

DONE

A WAV and an MP3 from a USB stick both play on the HP through its HD Audio codec, without the UI stuttering.

16 0.9.4 “Midas” — the trackpad, if it is I2C

SIZE	Medium, and conditional
DEPENDS	The measurement in section 25
UNLOCKS	A laptop that works without anything plugged into it
RISK	The release may turn out to be unnecessary. That is a good risk

GOAL. The HP’s built-in trackpad works. Whether this release exists at all is decided by one boot of usb -vv after 0.8.5, and that boot should happen as early as possible.

What the measurement shows	What happens
The trackpad enumerates as USB HID	0.8.5 already fixed it. This release is cancelled and the slot is returned to the schedule
It is I2C-HID behind Intel Serial-IO	This release is written: the LPSS I2C controller, then HID-over-I2C

WORK, if needed. The Intel LPSS I2C controller is a Synopsys DesignWare part behind a PCI function with an MMIO BAR — which the MMIO seam from 0.8.3.1 already makes testable. On top of it, the HID-over-I2C protocol: read the HID descriptor from the register the ACPI tables name, then input reports. **The awkward part is ACPI**: the descriptor address and the interrupt normally come from the DSDT, and this kernel has no ACPI parser. A minimal table walk to find one device is in scope; a general AML interpreter is not.

DECIDED

If the trackpad is I2C-HID, **the driver gets written**. RM-1002 listed it as out of 1.0 and that is reversed here: the difference between a laptop that works and one that needs a dongle permanently plugged into it is the difference between an operating system and a demonstration.

DONE

The pointer moves on the HP with nothing plugged into any port.

17 0.9.5 to 0.9.9 — stabilisation

Five releases with no new capability in them. This is the part of every plan that gets compressed when the end is in sight, and the part that decides whether 1.0 is a version number or a release.

Release	What it is
0.9.5	**Zero-warning build.** desktop.c alone had 17 warnings at 3400 lines. Then the static-analysis pass
0.9.6	**Fuzzing.** FAT16 and FAT32 structures, the TCP/IP receive path, TLS records, and the shell parser. Each gets a corpus and a crash budget of zero
0.9.7	**Soak and performance.** Days of uptime, not minutes — which is also the only realistic way to catch the idle panic. Memory growth measured, not assumed
0.9.8	**The hardware matrix.** Every machine available, every bus, every install path, written down as a table that ships with the release
0.9.9	**Feature freeze and the release candidate.** The manual and its screenshots regenerated from a running system; documentation gaps closed; the RC built from the source archive in an empty directory

The carried bugs, all of which must close here or earlier

- **The idle panic.** The laptop panicked once sitting at a shell prompt untouched. The broken IRQ frame pointer is the best candidate and is fixed; it has not been reproduced. The panic screen is truthful now, so if it recurs, take the EIP and run nm -n havendos.elf. **0.9.7's soak is the test that would find it.**
- **tests/test_fat16path.c still skips** when BOOTDISK is unset, so make test never exercises it. Give it a generated fixture or delete it — a suite that reports nothing is worse than no

suite. It also writes to the image it is pointed at, and once replaced a 631 KB kernel with a 2 KB test file.

- **The FAT16 dotfile bug**, if 0.9.1 did not already take it.
- **The security and login audit.** Credentials look plaintext in the tree. Whatever the answer is, 1.0 should state it rather than leave a reader to find out.

18 1.0 “Olympus”

- The six gates in section 22, each on hardware rather than in QEMU, with the AHCI qualification stated in the release notes.
- The standing ones: full suite green, the five build checks passing, the help checker consistent, and the build reproducible from the source archive in an empty directory.
- Every carried bug in section 17 closed, or written down as a known limitation with its reason — which is how this tree has always handled the ones it could not close.
- The manual and its screenshots regenerated from a running system.
- A changelog that says what 1.0 cannot do as plainly as what it can.

WHAT 1.0 IS NOT

1.0 is not the end of HavenDOS and it is not the browser. It is the point at which an operating system written by one person installs itself onto a modern laptop beside Windows, fills the screen, takes input from USB, reaches a network, and leaves everything else on the disk byte-identical. That is the thing worth shipping, and the web is the next thing after it.

19 1.1 “Aegis” and 1.2 “Janus” — the web

Written out here in full, because a thing deferred without a plan is a thing abandoned.

1.1 “Aegis” — modern TLS

SIZE	Large
DEPENDS	1.0, and a working network from 0.8.7
UNLOCKS	HTTPS that real servers accept — wget, curl, chat
RISK	Cryptographic code is correct or it is worthless

The tree offers one cipher suite, TLS_RSA_WITH_AES_128_CBC_SHA256. What is missing, in the order it has to be built:

1. **X25519** — field arithmetic modulo $2^{255}-19$ and a Montgomery ladder. Compact, and the single most valuable piece: it is what makes ECDHE possible, and no modern server will negotiate without it or an equivalent.
2. **AES-GCM** — counter mode over the existing AES, plus GHASH, which is a carry-less multiply in GF(2^{128}). Constant-time, table-free.
3. **HKDF and the TLS 1.3 key schedule** over the existing SHA-256.
4. **The TLS 1.3 handshake** — which is *simpler* than 1.2, not harder: one round trip, everything after ServerHello encrypted, no renegotiation, no compression, no CBC padding oracle to get wrong. The three padding bugs v0.7.5 fixed cannot recur because the construction is gone.
5. **Signature verification** — RSA-PSS and ECDSA on P-256, which is the part that actually makes a certificate mean something. The parser is already bounds-checked; what is missing is the verify.

6. **A trust store** — a small pinned set of root CAs compiled in, with an explicit, visible way to say “connect anyway” that is never the default. This has been open since v0.7.7.

> **NOTE**
> Test against real servers from day one, not against our own code. The existing TLS was broken against real servers in three separate ways — a record-splitting bug, a padding-semantics bug and a NewSessionTicket abort — and each only showed up against a server nobody here wrote.

1.2 “Janus” — the browser

SIZE	Large
DEPENDS	1.1. There is no point rendering pages it cannot fetch
UNLOCKS	The thing that was asked for
RISK	Scope. A browser is infinite unless its edges are drawn first

What it is: a text browser on the HavenDOS console grid. HTTP/1.1 and HTTPS, HTML parsed into a document tree, block and inline layout to the grid width, links numbered and followed by keyboard, history, bookmarks, and forms with GET and POST. Redirects, chunked encoding, gzip, and character-set handling that reuses the CP437 work from v0.7.15.

What it is not, and this is written down before a line is typed: no JavaScript, no CSS layout beyond display and a handful of inline hints, no images in the first release, no video, no fonts. The reference point is Lynx, not Chrome — and Lynx is genuinely useful, which is the argument for aiming there deliberately rather than arriving there disappointed.

Concurrency: none. The decision taken is that the fetch loop calls the same present-and-poll functions the rest of the system uses, so the screen stays live and ESC cancels. HavenDOS polls everything; a browser does not get to be the exception, and the scheduler that would make it one is section 24.

20 Cross-cutting work

One layer per concern, taught once

> **THE RULE**
>
> **When you add a bus, grep for every place that chooses one.** Three of the six bugs found on metal in 0.8.3.1 were one mistake: the installer has one block layer and the filesystem has another, and only one of them got taught. This plan adds **a USB bus** (0.8.6), **three network adapters** (0.8.7) and **a second filesystem format** (0.9.1). Each is the same trap.

The countermeasure is structural rather than diligent: 0.8.5 builds the USB core with two back-ends from birth, and 0.8.7 builds the NIC interface with three drivers from birth. An abstraction with one implementation is a wrapper and proves nothing; with two, the compiler enforces it.

The seams that make testing possible

- **include/io.h** — port I/O behind six functions, which is how PCI enumeration and UHCI are tested without booting anything.
- **include/mmio.h** — delivered in 0.8.3.1 and adopted by AHCI. **xHCI, HD Audio and the Intel I2C controller are all MMIO devices**, so all three are testable from the day they are written. This is the single most valuable thing 0.8.3.1 left behind.
- **The viewport layer** — 0.8.8 delivers it, and it is what makes 0.8.9 and 0.9.0 incremental rather than one 550-site commit.

The build checks stay, and grow

Check	What it refuses
check_efi_embed.py	A kernel with no UEFI loader in it, or only one copy
check_screen_width.py	A string literal written past column 80
check_boot_log.py	Invented boot output
check_help.py	A command that dispatches but is not documented
check_iso_version.py	An ISO whose kernel and version.txt disagree

Two more are worth adding as the plan proceeds: one that refuses a new block-device bus not registered everywhere a bus is chosen, and one that refuses an app registry entry with no ported flag.

21 Sequencing

Four hard prerequisites. Everything else can move.

This	cannot land before	because
Everything	0.8.4 Hermes	otherwise every release reaches the HP by reinstalling beside Windows
0.8.6 USB storage	0.8.5 the USB core and xHCI	bulk transfers need endpoints, which need a configuration walk
0.8.7 USB Ethernet	0.8.5 the USB core and xHCI	same reason; the two PCI NICs in that release have no such dependency
0.8.9 and 0.9.0 app port	0.8.8 the console and viewport	there is no larger grid to port to, and no way to not regress

SEQUENCING RULE, CARRIED FROM RM-1001 AND RM-1002

A write path does not land before the thing that protects it. NVMe write waited for the additive installer; USB mass-storage *write* waits for the same installer refusals to know about USB; the FAT32 formatter validates geometry before the first sector rather than at step 4 of 9. The refusals are the second line of defence, not the first.

Three releases float: **0.9.1 Gaia**, **0.9.2 Prometheus** and **0.9.3 Orpheus** depend on nothing but 0.8.6, and any of them can move earlier if the USB chain stalls. **0.8.8 Helios** depends on nothing at all and is the obvious thing to do while waiting on a dongle to arrive or a controller to come up.

22 Release gates

The six claims in section 3, each turned into something observable. 1.0 ships when all six pass on hardware, not in QEMU.

Claim	Gate	Lands at
Installs	Installs onto a GPT disk with another OS on it, from the installer UI, with no hand-editing	0.8.3 — done
Boots	The machine boots the installed system under UEFI with no removable media attached **and no boot entry added from another OS**	0.9.2 Prometheus
Fills the screen	The desktop and every app occupy the whole panel on a 1080p machine, at a grid chosen without being told	0.9.0 Hephaestus
Networks	ifconfig shows a DHCP address and BOOT Chat connects	0.8.7 Iris
Takes input	Keyboard and a pointing device both work on a machine with no PS/2 port	0.8.5, or 0.9.4 if the trackpad is I2C
Harms nothing	Every sector outside the installed partition is byte-identical before and after, proved by checksum, and the other operating system still boots	0.8.3 — done, and re-proved each release

THE QUALIFICATION, STATED RATHER THAN IMPLIED

AHCI will ship at 1.0 having never run on real silicon. There is no SATA machine available to this project. It is proved against the simulated controller in tests/test_ahci.c, a 32-check suite whose injection harness found 12 holes, and against QEMU's q35 — and q35 + if=ide is AHCI, not IDE, which is how it is exercised at all. The release notes say this in the same words. A reader with a SATA desktop is the obvious person to ask, and the Discord is the obvious place to ask them.

23 Deliberately not in 1.0

Left out	Why
Modern TLS and the browser	Not cut — moved. 1.1 and 1.2, written out in full. Binding them to 1.0 means neither ships
Wi-Fi	Signed firmware blobs per chipset and a full 802.11 MAC. The HP's MediaTek 14C3:7902 is exactly this problem, and a USB dongle is the honest answer at 1.0
Intel VMD / RST	A firmware setting solves it in seconds; a driver would take weeks and help only machines whose owner will not change that setting
SD, eMMC, SAS, hardware RAID	Each is a controller family of its own. "Every drive" is scoped to IDE, AHCI, NVMe, VirtIO and USB
NTFS	Reading a Windows partition is not needed in order to live beside one
Secure Boot	Requires a signed loader. Users can turn it off, and 1.0 does not need to solve it
Multitasking	Orthogonal to running on modern hardware, and large. The browser was the strongest argument for it and the polling decision removes that
64-bit	Invalidates every piece of 32-bit assembly and every driver's pointer assumptions, for no capability 1.0 claims

All four of the last entries are named in section 24 rather than left unmentioned, because an item with no plan is an item quietly dropped.

24 After 1.2 — the horizon

Not scheduled, and named on purpose. Each of these is larger than any single release in this document.

Next	What it is, honestly
Multitasking and processes	A preemptive scheduler, memory protection, real isolation. The thing that turns HavenDOS from an OS-shaped program into an operating system. Probably the first of these four to attempt, because everything else is easier on top of it
64-bit, long mode	More than 4 GB of RAM and a modern toolchain. Invalidates the bootstrap, the paging setup, every piece of assembly and every driver's assumption that a pointer fits a uint32_t — which is also what -no-pie on the driver suites exists to preserve today
Wi-Fi	One specific chipset with a firmware blob and an 802.11 MAC — probably *not* the HP's MT7902, which is new and poorly documented. Pick the chipset by what is documented, not by what is in the laptop

Next	What it is, honestly
Port shell, filesystem and network into HavenOS	The long-stated plan: HavenDOS’s userland becomes HavenOS’s, and the graphical system becomes the main project. Everything built in this document is built once and used twice, which is the argument for keeping the layers clean while building them

25 Open questions

Three, and each is one boot on the HP. Each changes the plan, so they are worth answering before the release they affect rather than during it. The fourth question in RM-1002 — whether the NVMe is hidden behind RST — is answered: it is visible, and the drive is in the boot log.

1. **What bus is the trackpad on?** One run of usb -vv after 0.8.5. USB means 0.8.5 already fixed it and section 16 is cancelled. I2C-HID behind Intel Serial-IO means that release is written.
2. **What does the firmware hand over with gfxpayload=keep?** Decides whether 0.8.8's auto-detect sees 1920x1080 or something the panel scales. Answerable today, before 0.8.8 starts.
3. **What chip is in the USB Ethernet adapter?** Not a measurement on the laptop but a decision about what to buy. It decides which of the three USB NIC drivers in 0.8.7 is written first, and the right answer is the one in the adapter already on the desk.

> **NOTE**
> Questions 1 and 2 can both be answered in a single boot, and question 2 needs no new code at all. Doing them in the week 0.8.4 is being built costs nothing and removes two unknowns from the two largest releases.

26 How all of it is verified

None of this is new. It is what the tree already enforces, applied to fourteen more releases.

- **make test green**, growing per release. 1657 checks across 26 suites today.
- **Every new driver gets an injection harness.** Real bugs put into the code, each of which the suite must notice. Every harness written so far has found holes — FAT32 missed 10 of 26, ESP 8 of 16, the formatter 6 of 21, the additive installer 9 of 21, AHCI 12 of 32. **A suite that passes on its first run is the least trustworthy result available.**
- **A driver is tested against a controller that executes its commands**, not a mock that returns success. tests/test_ahci.c and tests/test_nvmerw.c are the template.
- **Our code agreeing with itself is not evidence.** fsck.vfat, mtools, sgdisk, parted, ffmpeg and a real web server judge the results.
- **Byte-identity, not "it looks right."** Anything that writes to a disk is checked sector by sector, before and after.
- **A fixture a suite needs is generated by the build**, or the suite is decoration. One suite sat in the output with no result for three releases because its fixture never existed.
- **No invented output.** The boot-log checker enforces it in the boot path, and any new status screen is held to the same rule — which now includes usb -vv, ifconfig and the audio device list.
- **And, since 0.8.3.1: simulation answers only the questions you thought to ask.** Six bugs in one afternoon on metal says the remaining ones are in the places no harness models — bring-up state left by firmware, devices that exist but are the wrong kind, and paths nothing depends on until they fail. Every feature release in this document is followed onto hardware, and section 4 says what happens next.

THE SHORTEST VERSION OF THIS DOCUMENT

Make new versions reachable (0.8.4). Make USB work, because it is input, storage and network on the only real machine (0.8.5–0.8.7). Make the screen fill (0.8.8–0.9.0). Finish the edges (0.9.1–0.9.4). Prove it (0.9.5–0.9.9). Ship it (1.0). Then build the web (1.1–1.2).